



Push_swap

Parce que Swap_push n'est pas aussi naturel

Résumé:

Ce projet vous fera trier des données sur une pile, avec un ensemble limité d'instructions, en utilisant le plus petit nombre d'actions possibles. Pour réussir, vous devrez manipuler différents types d'algorithmes et choisir la solution la plus appropriée (parmi plusieurs) pour un tri optimisé des données.

Version: 8

Contenu

I	Avant-propos	2
II	Introduction	4
III	Objectifs	5
IV	Instructions communes	6
V	Partie obligatoire	8
V.1	Les règles	8
V.2	Exemple	9
V.3	Le programme "push_swap"	10
V.4	Référence	12
VI	Partie bonus	13
VI.1	Le programme "checker"	13
VII	Soumission et évaluation par les pairs	15

Chapitre I

Avant-prop

OS

- C

```
#include <stdio.h>

int main(void)
{
    printf("hello, world\n");
    return 0;
}
```

- ASM

```
cseg segment
assume cs:cseg, ds:cseg
org 100h
main proc
jmp debut
mess db 'Hello world!$'
debut:
mov dx, offset mess
mov ah, 9
int 21h
ret
main endp
cseg ends
end main
```

- LOLCODE

```
HAI
CAN HAS STDIO?
VISIBLE "HELLO WORLD!"
KTHXBYE
```

- PHP

```
<?php
    echo "Hello world!";
?>
```

- BrainFuck

```
+++++++[>+++++++>+++++++>+++><<<<-]
>+.,>+.+++++.,.+++,>+.,
<<+++++++>+.,>+.,-----,-----,>+,>
```


- C#

```
using System;

public class HelloWorld {
    public static void Main () {
        Console.WriteLine("Hello world!");
    }
}
```

- HTML5

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>Hello world !</title>
  </head>
  <body>
    <p>Hello World !</p>
  </body>
</html>
```

- YASL

```
"Hello world!"
print
```

- OCaml

```
let main () =
  print_endline "Hello world !"

let _ = main ()
```

Chapitre II

Introduction

Le projet **Push swap** est un projet d'algorithme très simple et très direct : les données doivent être triées

Vous avez à votre disposition un ensemble de valeurs entières, 2 piles, et un ensemble d'instructions pour manipuler les deux piles.

Ton but? Écrivez un programme en C appelé `push_swap` qui calcule et affiche sur la sortie standard le plus petit programme, constitué d'instructions du langage Push swap, qui trie les entiers reçus en arguments..

Facile?

Nous

verrons....

Chapitre

III

Objectifs

L'écriture d'un algorithme de tri est toujours une étape très importante dans le parcours d'un développeur. C'est souvent la première rencontre avec la notion de complexité.

Les algorithmes de tri et leur complexité font partie des questions classiques abordées lors des entretiens d'embauche. C'est probablement le bon moment pour examiner ces concepts puisque vous devrez y faire face à un moment donné.

Les objectifs d'apprentissage de ce projet sont la rigueur, l'utilisation du C et l'utilisation d'algorithmes de base.
En se concentrant particulièrement sur leur complexité.

Le tri des valeurs est simple. Les trier le plus rapidement possible est moins simple. D'autant que d'une configuration d'entiers à l'autre, la solution de tri la plus efficace peut différer.

Chapitre IV

Instructions communes

- Votre projet doit être écrit en C.
- Votre projet doit être rédigé conformément à la Norme. Si vous avez des fichiers/fonctions bonus, ils sont inclus dans le contrôle de norme et vous recevrez un 0 s'il y a une erreur de norme à l'intérieur.
- Vos fonctions ne doivent pas s'arrêter de manière inopinée (erreur de segmentation, erreur de bus, double libération, etc.) en dehors de comportements non définis. Si cela se produit, votre projet sera considéré comme non fonctionnel et recevra un 0 lors de l'évaluation.
- Tout l'espace mémoire alloué au tas doit être correctement libéré si nécessaire. Aucune fuite ne sera tolérée
- Si le sujet l'exige, vous devez soumettre un Makefile qui compilera vos fichiers sources vers la sortie requise avec les drapeaux -Wall, -Wextra et -Werror, utilisez cc, et votre Makefile ne doit pas établir de nouveau lien..
- Votre Makefile doit au moins contenir les règles \$(NAME), all, clean, fclean et concernant
- Pour donner des bonus à votre projet, vous devez inclure une règle bonus dans votre Makefile, qui ajoutera tous les différents en-têtes, bibliothèques ou fonctions interdites sur la partie principale du projet. Les bonus doivent être dans un fichier différent _bonus.{c/h} si le sujet ne précise rien d'autre. L'évaluation des parties obligatoires et bonus est effectuée séparément.
- Si votre projet vous permet d'utiliser votre libft, vous devez copier ses sources et son Makefile associé dans un dossier libft avec son Makefile associé. Le Makefile de votre projet doit compiler la bibliothèque en utilisant son Makefile, puis compiler le projet.
- Nous vous encourageons à créer des programmes de tests pour votre projet même si ce travail ne sera pas soumis et ne sera pas noté. Cela vous donnera l'occasion de tester facilement votre travail et celui de vos pairs. Vous trouverez ces tests particulièrement utiles lors de votre soutenance. En effet, lors de la soutenance, vous êtes libre d'utiliser vos tests et/ou les tests du pair que vous évaluez.
- Soumettez votre travail à votre référentiel git attribué. Seul le travail dans le dépôt git sera noté. Si Deepthought est chargé de noter votre travail, cela sera fait

Après vos évaluations par les pairs. Si une erreur se produit dans une section de votre travail lors de la notation de Deepthought, l'évaluation s'arrêtera..

Chapitre V

Partie

obligatoire

V.1 Les règles

- Vous disposez de 2 piles nommées a et b.
- Au début:
 - La pile a contient une quantité aléatoire de nombres négatifs et/ou positifs qui ne peuvent être dupliqués.
 - La pile b est vide.
- Le but est de trier par ordre croissant les numéros dans la pile a. Pour ce faire, vous disposez des opérations suivantes :

sa (swap a): Échangez les 2 premiers éléments en haut de la pile a.
Ne faites rien s'il n'y a qu'un ou aucun élément.

sb (swap b): Échangez les 2 premiers éléments en haut de la pile b.
Ne rien faire s'il n'y a qu'un ou aucun élément.

ss : sa et sb en même temps.

pa (push a): Prenez le premier élément en haut de b et placez-le en haut de a.
Ne rien faire si b est vide.

pb (push b): Prenez le premier élément en haut de a et placez-le en haut de b.
Ne faites rien si a est vide.

ra (rotate a): Augmentez tous les éléments de la pile a de 1.
Le premier élément devient le dernier..

rb (rotate b): Augmentez tous les éléments de la pile b de 1.
Le premier élément devient le dernier.

rr : ra et rb en même temps.

rra (reverse rotate a): Décaler tous les éléments de la pile a de 1.
Le dernier élément devient le premier.

rrb (reverse rotate b): Décalez tous les éléments de la pile b de 1.
Le dernier élément devient le premier.

rrr : rra et rrb en même temps.

V.2 Exemple

Pour illustrer l'effet de certaines de ces instructions, trions une liste aléatoire d'entiers. Dans cet exemple, nous considérerons que les deux piles grandissent à partir de la droite.

```
-----
Init a and b:
2
1
3
6
5
8
- -
a b
-----
Exec sa:
1
2
3
6
5
8
- -
a b
-----
Exec pb pb pb:
6 3
5 2
8 1
- -
a b
-----
Exec ra rb (equiv. to rr): 5
2
8 1
6 3
- -
a b
-----
Exec rra rrb (equiv. to rrr): 6
3
5 2
8 1
- -
a b
-----
Exec sa:
5 3
6 2
8 1
- -
a b
-----
Exec pa pa pa:
1
2
3
5
6
8
- -
a b
-----
```

Les entiers d'un sont triés en 12 instructions. Peux-tu faire mieux

V.3 Le programme "push_swap"

Nom du programme	push_swap
Remettre les fichiers	Makefile, *.h, *.c
Makefile	NAME, all, clean, fclean, re
Arguments	pile a : une liste d'entiers
Fonctions externes.	• read, write, malloc, free, exit • ft_printf et tout équivalent que VOUS avez codé
Libft autorisé	Oui
Description	Trier les piles

Votre projet doit respecter les règles suivantes ::

- Vous devez rendre un Makefile qui compilera vos fichiers sources. Il ne faut pas reconnecter.
- Les variables globales sont interdites.
- Vous devez écrire un programme nommé push_swap qui prend comme argument la pile formatée sous forme de liste d'entiers. Le premier argument doit être en haut de la pile (attention à l'ordre)
- Le programme doit afficher la plus petite liste d'instructions possible pour trier la pile
a, le plus petit nombre étant en haut.
- Les instructions doivent être séparées par un '\n' et rien d'autre.
- Le but est de trier la pile avec le moins d'opérations possible. Lors du processus d'évaluation, le nombre d'instructions trouvées par votre programme sera comparé à une limite : le nombre maximum d'opérations tolérées. Si votre programme affiche une liste plus longue ou si les nombres ne sont pas triés correctement, votre note sera de 0.
- Si aucun paramètre n'est spécifié, le programme ne doit rien afficher et renvoyer l'invite
- En cas d'erreur, il doit afficher "Erreur" suivi d'un '\n' sur l'erreur standard. Les erreurs incluent par exemple : certains arguments ne sont pas des entiers, certains arguments sont plus grands qu'un entier et/où il y a des doublons.


```
$> ./push_swap 2 1 3 6 5 8
sa
pb
pb
pb
sa
pa
pa
pa
$> ./push_swap 0 one 2 3
Error
$>
```

Pendant le processus d'évaluation, un binaire sera fourni afin de vérifier correctement votre programme

Cela fonctionnera comme suit:

```
$> ARG="4 67 3 87 23"; ./push_swap $ARG | wc -l
6
$> ARG="4 67 3 87 23"; ./push_swap $ARG | ./checker_OS $ARG
OK
$>
```

Si le programme checker_OS affiche "KO", cela signifie que votre push_swap a proposé une liste d'instructions qui ne trie pas les nombres.



Le programme checker_OS est disponible dans les ressources du projet sur l'intranet.

Vous pouvez trouver une description de son fonctionnement dans la partie bonus de ce document.

V.4 Référence

Pour valider ce projet, vous devez effectuer certains tris avec un nombre minimal d'opérations :

- Pour une validation minimaliste (donc un maximum de 80 sur votre note), vous devez être capable de trier 100 nombres aléatoires en moins de 700 opérations. Il est à noter que nous parlons d'une moyenne, mais généralement, vous devriez toujours avoir moins de 700 opérations à réaliser au maximum..
- Pour une validation maximale du projet et ainsi pouvoir obtenir les bonus, vous devez remplir la première étape ci-dessus, mais aussi pour 500 nombres aléatoires, il ne doit pas y avoir plus de 5500 opérations.

Tout cela sera vérifié lors de votre évaluation.

Chapitre VI

Partie bonus

Ce projet laisse peu de place à l'ajout de fonctionnalités supplémentaires en raison de sa simplicité. Cependant, que diriez-vous de créer votre propre vérificateur ?



Grâce au programme checker, vous pourrez vérifier si la liste d'instructions générée par le programme push_swap trie correctement la pile.

VI.1 Le programme "checker"

Nom du programme	checker
Remettre les fichiers	*.h, *.c
Makefile	bonus
Arguments	stack a: A list of integers
Fonctions externes	• read, write, malloc, free, exit • ft_printf et tout équivalent que VOUS avez codé
Libft autorisé	Oui
Description	Exécuter les instructions de tri

- Écrivez un programme nommé checker qui prend comme argument la pile formatée sous forme de liste d'entiers. Le premier argument doit être en haut de la pile (faites attention à l'ordre). Si aucun argument n'est donné, il s'arrête et n'affiche rien.
- Il attendra ensuite et lira les instructions sur l'entrée standard, chaque instruction sera suivie de '\n'. Une fois toutes les instructions lues, le

programme doit les exécuter sur la pile reçue en argument.

- Si après avoir exécuté ces instructions, la pile a est effectivement triée et la pile b est vide, alors le programme doit afficher "OK" suivi d'un '\n' sur la sortie standard.
- Dans tous les autres cas, il doit afficher "KO" suivi d'un '\n' sur la sortie standard.
- En cas d'erreur, vous devez afficher "Erreur" suivi d'un '\n' sur l'erreur standard. Les erreurs incluent par exemple : certains arguments ne sont pas des entiers, certains arguments sont plus grands qu'un entier, il y a des doublons, une instruction n'existe pas et/ou est mal formatée.

```
$>./checker 3 2 1 0
rra
pb
sa
rra
pa
OK
$>./checker 3 2 1 0
sa
rra
pb
KO
$>./checker 3 2 one 0
Error
$>./checker "" 1
Error
$>
```

Vous N'ÊTES PAS obligé de reproduire exactement le même comportement que le binaire fourni. Il est obligatoire de gérer les erreurs mais c'est à vous de décider comment vous souhaitez analyser les arguments.

La partie bonus ne sera évaluée que si la partie obligatoire est PARFAITE. Parfait signifie que la partie obligatoire a été intégralement réalisée et fonctionne sans dysfonctionnement. Si vous n'avez pas satisfait à TOUTES les exigences obligatoires, votre partie bonus ne sera pas du tout évaluée.

Chapter VII

Soumission et peer-evaluation

Remettez votre devoir dans votre référentiel Git comme d'habitude. Seuls les travaux contenus dans votre dépôt seront évalués lors de la soutenance. N'hésitez pas à vérifier les noms de vos fichiers pour vous assurer qu'ils sont corrects.

Ces affectations n'étant pas vérifiées par un programme, n'hésitez pas à organiser vos fichiers comme vous le souhaitez, à condition de remettre les fichiers obligatoires et de respecter les exigences.



```
file.bfe:VABB7y09xm7xWXR0eASmsgnY0o0sDMJev7zFHhwQS8mvM8V5xQQp  
Lc6cDCFXDWTiFzZ2H9skYkiJ/DpQtnM/uZO
```